Chapter 14: Vector Space Classification

https://drive.google.com/file/d/1sWvloE15gwzy-4MfuBntBKjSYAKbr2Fv/view
https://drive.google.com/file/d/1uB9leiG-HEAtEX9IgSXdg_DFDWvUVAT4/view

1. Vector Space Model for Classification

- each document is represented as a feature vector in a high-dimensional space (**one dimension per vocabulary term**).

💡 Contiguity Hypothesis:

> **Documents in the same class form** a **contiguous region in vector space,** and **regions of different classes do not overlap.**
> This **allows a classifier to draw decision boundaries.**

> Classification Pipeline

- Corpus – the classification dataset
- Pre-processing – tokenisation, stemming, stopword removal
- Feature selection & weighting – TF-IDF, MI, χ^2
- Similarity/distance function
- Classification algorithm
- Performance evaluation

2. Rocchio's Algorithm

- Rocchio's algorithm adapts the relevance feedback technique for classification.
- It **divides** the **vector space into regions centred on class centroids** (prototypes)
- simple and efficient
- **inaccurate** if classes are **not approximately spheres** with similar radii.

› Algorithm:

TRAINROCCHIO($\mathbb{C}, \mathbb{D}$)

for each $c_j \in \mathbb{C}$

do $D_j \leftarrow \{d : \langle d, c_j \rangle \in \mathbb{D}\}$

$\quad \vec{\mu}_j \leftarrow \frac{1}{|D_j|} \sum_{d \in D_j} \vec{v}(d)$

return $\{\vec{\mu}_1, \ldots, \vec{\mu}_J\}$

APPLYROCCHIO($\{\vec{\mu}_1, \ldots, \vec{\mu}_J\}, d$)

return $\arg\min_j |\vec{\mu}_j - \vec{v}(d)|$

For each class c_j, compute the **centroid** (centre of mass) of all training documents in that class:

$$\vec{\mu}_j \leftarrow \frac{1}{|D_j|} \sum_{d \in D_j} \vec{v}(d)$$

Assign a new document d to the class whose **centroid is closest**:

$$\arg\min_j |\vec{\mu}_j - \vec{v}(d)|$$

› Worked Example – Rocchio

Doc	Words	Class	Vector (beijing, chinese, japan, macao, shanghai, tokyo)
1	Chinese Beijing Chinese	c	‹1, 2, 0, 0, 0, 0›
2	Chinese Chinese Shanghai	c	‹0, 2, 0, 0, 1, 0›
3	Chinese Macao	c	‹0, 1, 0, 1, 0, 0›
4	Tokyo Japan Chinese	j	‹0, 1, 1, 0, 0, 1›
5 (test)	Chinese Chinese Chinese Tokyo Japan	?	‹0, 3, 0, 0, 0, 1›

- μ^c = (1/3)(d1 + d2 + d3) = ‹1/3, 5/3, 0, 1/3, 1/3, 0›
- μ□ = d4 = ‹0, 1, 1, 0, 0, 1›
- Distance(d5, μ^c) = 5

✔ Distance to class c

$$d(doc5, \mu_c)$$

Compute difference:

$$(0-1/3,\ 3-5/3,\ 0-0,\ 0-1/3,\ 0-1/3,\ 1-0)$$
$$=(-1/3,\ 4/3,\ 0,\ -1/3,\ -1/3,\ 1)$$

Square & sum:

$$=(1/9+16/9+0+1/9+1/9+1)\approx 5$$

- Distance(d5, μ□) = 4

Result: d5 is classified as class j (closer to j centroid)

› Weaknesses of Rocchio Classification

- **Inaccurate** when classes are not **approximately spherical** in shape
- Fails when classes have **similar radii but are non-convex or multimodal**
- A document **can only be assigned to one class** (single prototype per class)

3. k-Nearest Neighbour (kNN) Classification

- kNN is an **instance-based (lazy) learning method.**
- It **does not build an explicit model during training** – instead it **memorises** the **entire training set** and **classifies at query time.**
- **Memory based method**

› Algorithm

Training: Simply **store all training examples and their feature vectors.** No model is built.

Testing for document d:

- Compute **cosine similarity between d** and **all training documents**
- **Find** the **k most similar** (**nearest**) **neighbours**
- Return the **majority class among those k neighbours**

› kNN Example

	Doc	Words	Class
Training	1	Chinese Beijing Chinese	c
	2	Chinese Chinese Shanghai	c
	3	Chinese Macao	c
	4	Tokyo Japan Chinese	j
Test	5	Chinese Chinese Chinese Tokyo Japan	?

Dictionary:

$$\langle \text{beijing, chinese, japan, macao, shanghai, tokyo} \rangle$$

Vectors:

Doc	Vector	Class
d1	⟨1,2,0,0,0,0⟩	c
d2	⟨0,2,0,0,1,0⟩	c
d3	⟨0,1,0,1,0,0⟩	c
d4	⟨0,1,1,0,0,1⟩	j
d5	⟨0,3,0,0,0,1⟩	? (test)

Using the same Chinese/Japan dataset:

$$\cos(\theta) = \frac{d_i \cdot d_j}{||d_i|| \, ||d_j||}$$

- Cos(d1, d5) = 0.848 → class c
- Cos(d2, d5) = 0.848 → class c
- Cos(d3, d5) = 0.424 → class c
- Cos(d4, d5) = 0.953 → class j

- 1-NN (k=1): d5 → **class j** (most similar single neighbour is d4)
- 3-NN (k=3): d5 → **class c** (majority of top 3 are class c: d1, d2, d4 → 2c vs 1j, but d4 is the 1st, d1 & d2 also top 3)

Advantages of kNN

- Simple, intuitive, and easy to implement
- **Only one hyperparameter (k) to tune**
- No assumptions about data distribution
- Naturally handles multiclass problems

Disadvantages of kNN

- **Sensitive to the choice of k**, distance function, and noisy data
- **Lazy learner** – no precomputed model (all computation at test time)
- **No training cost**
- **very large computation and storage cost** for large datasets
- All **features treated equally** (noisy features degrade performance)

‹ Food for Thoughts ›

Q4. Rocchio's Algorithm for Text Classification

Rocchio's algorithm computes a prototype (centroid) for each class from the training documents and classifies new documents by finding the closest centroid.

Modification: The algorithm can be adapted by weighting training documents (e.g. by TF-IDF) before computing centroids, which gives more important terms more influence in the centroid computation.

Q5. Weaknesses of Rocchio Classification

- Assumes classes are approximately spherical – fails for non-convex or irregular shapes
- One centroid per class cannot capture multimodal distributions
- Sensitive to outliers which distort the centroid
- Cannot handle overlapping classes well

Q6. Benefits of kNN for Text Classification

- Simple and easy to implement – no training phase required
- No assumption about class shape or distribution
- Naturally handles multiclass problems
- Can be updated by simply adding new training examples
- Works well with sufficient training data

Q7. Multinomial NB – Classification Example

Split	Doc ID	Words in Document	In c = China?
Training	1	Taipei Taiwan	yes
Training	2	Macao Taiwan Shanghai	yes
Training	3	Japan Sapporo	no
Training	4	Sapporo Osaka Taiwan	no
Test	5	Taiwan Taiwan Sapporo	?

Part (a): Class Prior Probabilities

- P(China = yes) = 2/4 = 0.5
- P(China = no) = 2/4 = 0.5

Part (b): Conditional Probabilities (Multinomial NB with Laplace smoothing)

Vocabulary V = {Taipei, Taiwan, Macao, Shanghai, Japan, Sapporo, Osaka} → |V| = 7

Class YES (China): Words are Taipei, Taiwan, Macao, Taiwan, Shanghai → total tokens = 5

- P(Taiwan | yes) = (2+1)/(5+7) = 3/12 = 1/4
- P(Sapporo | yes) = (0+1)/(5+7) = 1/12
- P(Taipei | yes) = (1+1)/(5+7) = 2/12 = 1/6
- P(Macao | yes) = (1+1)/(5+7) = 2/12 = 1/6
- P(Shanghai | yes) = (1+1)/(5+7) = 2/12 = 1/6
- P(Japan | yes) = (0+1)/(5+7) = 1/12
- P(Osaka | yes) = (0+1)/(5+7) = 1/12

Class NO: Words are Japan, Sapporo, Sapporo, Osaka, Taiwan → total tokens = 5

- P(Taiwan | no) = (1+1)/(5+7) = 2/12 = 1/6
- P(Sapporo | no) = (2+1)/(5+7) = 3/12 = 1/4
- P(Japan | no) = (1+1)/(5+7) = 2/12 = 1/6
- P(Osaka | no) = (1+1)/(5+7) = 2/12 = 1/6
- P(Taipei | no) = (0+1)/(5+7) = 1/12
- P(Macao | no) = (0+1)/(5+7) = 1/12
- P(Shanghai | no) = (0+1)/(5+7) = 1/12

Part (c): Classify Test Document d5 = {Taiwan, Taiwan, Sapporo}

Score(yes) = log P(yes) + log P(Taiwan|yes) + log P(Taiwan|yes) + log P(Sapporo|yes)

- = log(0.5) + log(1/4) + log(1/4) + log(1/12)
- = -0.693 + (-1.386) + (-1.386) + (-2.485) = -5.950

Score(no) = log P(no) + log P(Taiwan|no) + log P(Taiwan|no) + log P(Sapporo|no)

- = log(0.5) + log(1/6) + log(1/6) + log(1/4)
- = -0.693 + (-1.792) + (-1.792) + (-1.386) = -5.663

Result: Score(no) = -5.663 > Score(yes) = -5.950, so d5 is classified as NOT China (no).